Сервис Python API / worker

Описание

Python 3.12, FastAPI/Pydantic 2. API проектов использует модуль Романа:

sync SQLAlchemy 2 + psycopg 3, Alembic, PostgreSQL платформы. Runs API и отдельный

worker управляют pipeline; generated product использует независимый C# backend.

Ссылки

Локальный Swagger: http://127.0.0.1:8000/docs; контракт —

../contracts/platform-openapi.json, методы — platform-api.md.

Prod/Stage URL не назначены; рабочий preview пока не заявляется.

Требования к ресурсам

Python 3.12 и PostgreSQL, диск под generated (до 24 MiB файлов продукта на попытку

по текущим лимитам плюс LLM inputs/outputs и отчёты). Измеренных RAM/CPU/IO лимитов

ещё нет; не подставляются цифры из примеров регламента. Ограничения runner задаёт Дмитрий.

Отказоустойчивость

P0: один worker. DB commit на создание проекта; RunRepository должен атомарно

забирать queued run. Heartbeat и stale recovery предусмотрены контрактом,

автоматического resume нет. При рестарте API данные проектов остаются в PostgreSQL.

Репликация и подтверждённый RTO/RPO не реализованы.

Предельные ограничения работы сервиса

Название проекта 1–120 символов, requirements 20–50000. Один активный run/project

должен обеспечиваться DB repository. Файлы продукта: до 500, до 1 MiB каждый,

дерево до 24 MiB; LLM response до 8 MiB. Нагрузочная пропускная способность не измерена.

Работа сервиса

HTTP возвращает созданный run без ожидания Qwen. Отдельный worker получает run

из repository и сохраняет шаги. Подключены projects, SQL RunRepository и Qwen factory. Qwen требует ENV;

validator пока отсутствует. Успешная полная генерация не подтверждена.

Авторизация

P0 — закрытая командная demo, без публичной регистрации и multi-tenancy.

Встроенного JWT/RBAC сейчас нет. Доступ ограничивается окружением; реальный

auth нельзя заявлять выполненным. API/worker не получают Docker socket.

Описание окружений

Test: временные SQLite в Python-тестах; контейнерный PostgreSQL для интеграционной

проверки, если она отмечена успешной в handoff. Local: Compose из docker-integration.md.

Stage/Prod: адреса и процедуры пока не предоставлены, не считаются развёрнутыми.

Сборка

platform/Dockerfile, root context. requirements.txt экспортирован из uv.lock

с hashes, pip require-hashes; запуск от UID 10001. Исходный Docker-шаблон Дмитрия

сохранён в истории ветки. Генерируемый код в этом образе не собирается/не исполняется.

Деплой

Локальная платформа: database healthy → migrate completed → api.

Worker — отдельный профиль pipeline. Deployment generated product — adapter Дмитрия;

реального cloud/prod deployment здесь нет. Инструкция: docker-integration.md.

Конфигурация

DATABASE_URL — PostgreSQL SQLAlchemy URL, RWB_RUNTIME_FACTORY — реальная

factory для runs/Qwen/runner, RWB_GENERATED_ROOT/RWB_TEMPLATES_ROOT — каталоги,

RWB_CORS_ORIGINS — JSON origins. Остальные лимиты — app/config.py.

Секреты только в ENV/.env вне Git. Не выводить URL с credentials в ошибки/логи.

Переменные окружения для CI/CD

CI команды пока у Дмитрия. Требуются секреты окружения БД и адаптеров;

Qwen key не передаётся продукту, GitLab token остаётся у publisher.

Конкретные имена Qwen/publisher ENV согласуются с владельцами, не придуманы здесь.

Мониторинг

/health: 200 при доступности repository/таблицы projects, иначе 503.

Это readiness API, а не признак готовности всего pipeline. Worker пишет безопасные

ошибки и heartbeat через repository. Zabbix/Prometheus здесь не подключены.

Логирование

STDOUT/STDERR контейнеров, названия классов ошибок и безопасные codes.

Raw provider body не логируется. Сбор в Elasticsearch/Kibana требует инфраструктуры

коллег; наличие такой системы из корпоративного примера не заявляется.

Сетевая связность

API опубликован на localhost:8000; БД внутри Compose network без host port.

Qwen доступен из VM организаторов согласно уточнению 30.10 16:17.

Сеть изолированного generated-code runner настраивается отдельно.

Доступы к сервису

Команда demo через локальный порт/согласованный доступ VM. Нет утверждённых

корпоративных подсетей, ролей доступа или процедур удалённой эксплуатации.

Объединённая платформа: запуск и границы проверки

Дата: 31.10.2026. Локальная ветка integration/complete-stack.

База integration/local-stack cbc6b57, интегрирована integration/roman-worker 84e775e.

В удалённый репозиторий изменения не отправлялись.

Состав

| Компонент | Путь | Состояние |

| --- | --- | --- |

| UI | apps/hackathon-rwb-angular | Production build и 12 тестов прошли |

| API | platform/app/api | Projects, runs, events, artifacts, ZIP |

| SQL | platform/app/integrations/sql_repository.py | Миграции до 0006; SQLite integration tests |

| Worker | platform/app/worker.py | SQL очередь, heartbeat, stale recovery; Qwen factory |

| Qwen | platform/app/llm | Нет credentials в среде проверки; реальный вызов заблокирован |

| Templates | templates | Backend, admin, PWA, PostgreSQL; требуется код модели |

| Runner | порт ValidationRunner | Реализация не обнаружена, validator=None; package блокируется |

| Compose | compose.yaml + compose.backend.yaml | Web, API, migrations, DB; worker в профиле pipeline |

Проверены ветки Docker 8c6af4c, backend b37c702, database_backend 4d091d7,

feature/frontend d24363a, incorrect-frontend 84e6766, main 7477da3.

SQL-изменения database_backend уже входят в roman-worker; повторный перенос не нужен.

Подготовка и запуск

Из корня репозитория, Docker Compose с поддержкой include:

bash

cp platform/.env.example platform/.env

PowerShell: Copy-Item platform/.env.example platform/.env

Установить POSTGRES_PASSWORD; Qwen-параметры получить у владельца доступа.

docker compose --env-file platform/.env up --build -d

UI: http://localhost:4200. Swagger: http://127.0.0.1:8000/docs.

Nginx проксирует /api/ к api:8000, поэтому браузеру не требуется Docker DNS.

Миграции выполняются до API; данные остаются в именованных volumes.

Не применять down -v к данным команды.

bash

docker compose --env-file platform/.env run --rm api python -m scripts.qwen_smoke

docker compose --env-file platform/.env --profile pipeline up --build -d

docker compose --env-file platform/.env logs --tail=100 worker api

docker compose --env-file platform/.env restart api worker database

Worker включать после настройки Qwen. До подключения настоящего ValidationRunner
полный run не сможет пройти validation/package даже при доступном Qwen.
Фабрика runtime: app.integrations.database_runtime:create_runtime; расширять её
реальным изолированным runner по platform/app/ports.py. Успех нельзя заменять
фиктивным ValidationReport. Runner не должен получать ключ Qwen, Docker socket
или работать непосредственно в неизменяемом generated workspace.

Qwen и окружения

QWEN_BASE_URL, QWEN_MODEL, QWEN_API_KEY и QWEN_API_PATH берутся из .env.
API path по умолчанию /chat/completions; проверить требования провайдера.
Для SSH-туннеля на Windows хост использует 127.0.0.1, контейнер — host.docker.internal.
Для доверенного HTTP-туннеля включить QWEN_ALLOW_DOCKER_HOST_HTTP=true.
На Linux loopback-only SSH listener может быть недоступен через host-gateway:
проверить bind-адрес и firewall. Не выставлять туннель публично.
Прямой 10.10.10.10:4000 здесь не проверялся. Секреты не включать в логи/архив.

Проверки и ограничения

Python tests включают настоящие SQLAlchemy/Alembic операции на временной SQLite,
но Qwen и runner в них заменены test doubles. Это не доказательство PostgreSQL
SKIP LOCKED, работы модели, .NET-сборки или запуска сгенерированного продукта.
Сборка Angular template проверяется в отдельной временной копии с тестовым
пустым generatedRoutes: проверяет host, а не доменные экраны модели.

В среде этой работы нет Docker, PostgreSQL server и .NET SDK; .env не передан.

Реальный Qwen smoke завершился qwen_not_configured до сетевого запроса.

Нет реального ID успешной генерации. Сквозной пользовательский сценарий не подтверждён.

Для завершения нужны окружение Docker/.NET, доступ Qwen и реализация runner.

bash

cd platform

python -m pip install --require-hashes -r requirements-dev.lock.txt

python -m pytest -q

python -m app.export_contracts

cd ../apps/hackathon-rwb-angular

npm ci --ignore-scripts --no-audit --no-fund

npm run build

npm test -- --watch=false

Проверка на машине команды

Создать проект через UI, запустить run, сохранить ID; проверить events, heartbeat,

появление артефактов и terminal status. Пока runner не подключён, ожидается failed,

а не готовый продукт. После его подключения проверить каждый mandatory check,

скачать ZIP, собрать и запустить продукт, проверить сценарий требований.

При отключении Qwen run должен завершаться ошибкой; после restart данные проекта

и история run должны остаться. Проверку конкурирующего claim выполнить на PostgreSQL.